

多模态 Agent Post-Training 项目

从零到一的完整指南

2026 年 6 月 3 日

摘要

本文档以初学者友好的方式，完整记录多模态 Agent Post-Training 项目从零搭建到当前状态的每一个决策、每一次失败、每一处优化。项目目标：训练 Qwen2.5-VL 视觉语言模型在收到“图片 + 问题”后，自动按 vision_parse → retrieve_docs/python_exec → final_answer 的多步工具调用链完成任务。全文按数据工程、模型训练、推理评测、Agent 优化四条主线展开，结合每一份代码文件解读其设计动机、解决的问题、实际效果，以及失败尝试背后的深层原因。

目录

1	项目概述：我们要做什么	3
1.1	一句话定义	3
1.2	核心约束为什么这么严格	3
1.3	项目代码结构总览	3
2	损失函数设计：从 SFT 到 RL 的完整演化	4
2.1	SFT 损失：模仿正确答案	4
2.2	DPO 损失：偏好比较（已弃用）	4
2.3	RL Reward 函数：多维度打分	5
2.4	REINFORCE 损失：奖励引导的梯度	5
2.5	KL 惩罚：安全绳	6
2.6	完整 RL 损失	6
3	概念速查：10 分钟理解所有术语	7
4	推理系统：agent/ 目录详解	7
4.1	agent/vlm.py — 模型的加载和推理	8
4.2	agent/parser.py — 解析模型输出	8
4.3	agent/tools.py — 三个工具的实现	9
4.3.1	vision_parse — 视觉解析	9
4.3.2	retrieve_docs — 规则文档检索	9
4.3.3	python_exec — 安全数学计算	9
4.4	agent/prompts.py — 提示词模块	9

4.5	agent/runtime.py — 多轮 Agent 执行引擎	10
5	训练系统: train/ 目录详解	11
5.1	train/train_sft_lora.py — SFT LoRA 训练	11
5.2	train/train_dpo_lora.py — DPO 偏好训练 (已弃用)	11
5.3	train/train_rl_custom.py — 自研 RL 训练 (当前版本)	12
6	评测系统: eval/ 目录详解	13
6.1	eval/run_vlm_eval.py — 评测主脚本	13
6.2	评测数据格式	13
7	数据工程: 完整的构建管线	15
7.1	数据全景	15
7.2	种子数据处理管线	15
7.3	外部数据集转换	15
7.4	真实 VLM 数据重建	16
7.5	数据增强脚本 (效果有限的尝试)	16
7.6	数据资源总览	16
8	完整的优化历程	18
8.1	阶段 1: 从零搭建 (V2 之前)	18
8.2	阶段 2: Mock 数据时代的达成 (V2)	18
8.3	阶段 3: 过度工具调用 (V3)	18
8.4	阶段 4: 真实 VLM 替代 Mock—质的飞跃 (V4)	18
8.5	阶段 5: 数据增强失败 (V5/V6/V8)	18
8.6	阶段 6: DPO 崩溃 (三次)	19
8.7	阶段 7: 7B 迁移	19
8.8	阶段 8: RL 探索	19
9	当前状态与最优模型	20
10	附录 A: 损失曲线可视化与分析	21
10.1	A.1 SFT 训练损失: V4 vs V7B	21
10.2	A.2 GRPO v1: reward 天花板与误导性 loss	21
10.3	A.3 GRPO v2: 缓存噪声导致的 loss 爆炸	22
10.4	A.4 reward 天花板的可视化	23
10.5	损失曲线总体诊断	23
11	附录 B: 完整模型版本对照表 (17 版)	23
12	附录 C: Eval 标签修正记录	24
13	附录 D: 图片处理管线	25

1 项目概述：我们要做什么	3
14 附录 E：GRPO 四轮算法迭代	25
15 核心教训	26
16 附录 F：完整代码引用索引	26
17 后续方向	28

1 项目概述：我们要做什么

1.1 一句话定义

给模型一张发票照片和问题”这张发票缺少必填字段吗？”，模型应该自己知道：(1) 先调 OCR 看清发票上的字，(2) 查规则文档找必填字段清单，(3) 对比后给出有依据的结论。

这不是简单的”看图回答问题”—模型需要自己决策每一步该调哪个工具，而且每轮只能做一件事。

1.2 核心约束为什么这么严格

每轮模型只能输出一个 `<tool_call>` 或一个 `<final_answer>`。这个约束不是我们故意为难模型—它是”工具调用”的本质：一次只能做一个动作，做了之后等结果，根据结果决定下一步。

这个约束是整个项目最重要的设计决策，也是后来几乎所有优化失败的根源。因为大部分 LLM 优化方法（DPO、RL、数据增强）都假设模型输出是”一段自由文本”，而我们要求的是”严格格式的单步骤”。

1.3 项目代码结构总览

项目根目录在 `/root/miniconda3/bin/multimodal-agent-posttraining(服务器)/`，有六个子目录：

- `agent/` — 推理核心：6 个 py 文件，负责模型的加载、推理、工具执行、输出解析
 - `train/` — 训练引擎：SFT LoRA 训练、DPO 偏好训练、自研 RL 训练
 - `eval/` — 评测系统：50 条评测样本的自动评测和指标计算
 - `scripts/` — 数据构建管线：25 个 py 脚本，负责数据的生成、转换、增强、分析
 - `data/` — 数据存储：SFT 训练数据、DPO 偏好数据、评测数据、规则文档、图片资源
 - `paper/` — 项目文档：LaTeX 论文和报告
- 外部存储（不在项目目录内）：
- `/root/autodl-tmp/models/` — 基座模型：Qwen2.5-VL-3B-Instruct(7.1GB) + 7B(16GB)
 - `/root/autodl-tmp/multimodal-agent-lora/` — 训练好的 LoRA 权重
 - `/root/autodl-tmp/multimodal-agent-data/` — 外部数据集图片 (ChartQA/DocVQA/CORD, 已 resize)

2 损失函数设计：从 SFT 到 RL 的完整演化

这一节专门讲”模型怎么知道自己学得好不好”—即每个训练阶段的损失函数（loss function）。损失函数是整个项目的灵魂：它定义了”什么是好行为”。后面的所有优化失败，归根结底都是损失函数和任务不匹配。

2.1 SFT 损失：模仿正确答案

SFT（Supervised Fine-Tuning）的损失是最简单的交叉熵（Cross-Entropy）：

$$\mathcal{L}_{SFT} = -\frac{1}{N} \sum_{i=1}^N \log P(y_i | x, y_{<i})$$

白话解释：对于训练数据中的每个 token，计算模型预测”正确 token”的概率，取负对数。概率越高，loss 越低。模型的目标就是不断提高”正确答案”的概率。

代码实现（train_sft_lora.py 的 QwenVLSFTCollator）：

```
# 只有assistant输出的token才参与loss计算
# prefix（用户消息、工具结果、之前的assistant输出）被mask掉
labels[row_index, :prefix_len] = -100 # -100 = 不计算loss
```

为什么 prefix 要 mask：我们只希望模型学会”在这个上下文中应该输出什么”，而不是让它”重新学习”所有的上下文。如果 prefix 也算 loss，模型会浪费大量计算在复制已有的对话史上。

SFT 的优点：简单、稳定、收敛快。**SFT** 的缺点：模型只能模仿训练数据中的行为，不能超越。如果数据中”直接回答”和”调工具后回答”的比例出了问题，模型就学歪了。

2.2 DPO 损失：偏好比较（已弃用）

DPO 的目标不是”模仿正确答案”，而是”A 比 B 好”：

$$\mathcal{L}_{DPO} = -\log \sigma \left(\beta \cdot \left(\log \frac{\pi_{\theta}(y_{chosen}|x)}{\pi_{ref}(y_{chosen}|x)} - \log \frac{\pi_{\theta}(y_{rejected}|x)}{\pi_{ref}(y_{rejected}|x)} \right) \right)$$

白话解释：chosen（好答案）和 rejected（坏答案）各算一个”得分”—模型对它的 log-prob 除以参考模型对它的 log-prob。如果 chosen 的得分明显高于 rejected，loss 就低。 β （通常 0.1）控制偏好强度， σ 是 sigmoid 函数（把任意值压到 0-1 之间）。

在这个项目里为什么失败：

关键问题： $\pi_{\theta}(y|x)$ 中的 y 是什么？

在标准 DPO 中， y 是完整的回答。但我们的任务是多轮的—每次对话包含 3-5 轮交替的 assistant 输出和 tool 结果。DPO 的代码实现（compute_log_prob）只取了最后一条 assistant 消息：

```
# train_dpo_lora.py: 只计算最后一条assistant消息的log-prob
assistant_idx = last_assistant_message(messages)
prefix = messages[:assistant_idx] # 前面的全部内容
full = messages[:assistant_idx + 1] # 包含最后的assistant

# 只优化 P(最后的assistant | 前面的全部上下文)
```

这导致 DPO 只优化最终答案，不优化工具调用步骤。当偏好数据是单轮的时候（user → final_answer），模型学到的就是”直接从问题跳到答案”——和 SFT 教的”先调工具再回答”直接冲突。

因此：DPO 在标准对话任务中有效，但在多轮工具调用任务中与 SFT 的格式纪律产生根本性冲突。

2.3 RL Reward 函数：多维度打分

REINFORCE/GRPO 不用固定的”正确答案”，而是用 reward 函数实时打分。模型自己生成 rollout，然后 reward 函数评判”这次做得好不好”。

当前 reward 函数（train_rl_custom.py 的 compute_reward）：

```
def compute_reward(result, gold):
    reward = 0.0
    # 1. 格式正确性（最高权重，-2表示绝不容忍格式错误）
    if result.status == "parse_error":    reward -= 2.0
    # 2. 工具选择（+1正确调用，-0.5漏调，+0.5正确不调）
    if needs_tools and tools_correct:    reward += 1.0
    elif needs_tools and tools_wrong:    reward -= 0.5
    elif no_tools_needed and no_tools:    reward += 0.5
    # 3. 答案质量（+0.5包含关键词/数值）
    if keywords_in_answer:                reward += 0.5
    # 4. 拒答正确性（+0.5该拒答时拒了）
    if refusal_correct:                   reward += 0.5
    # 5. 冗余工具惩罚（-0.3该不调时调了）
    if no_tools_needed and tools_called:  reward -= 0.3
    return reward # 范围约 [-2.8, +2.5]
```

权重设计逻辑：

- 格式扣分最重 **(-2.0)**——因为没有格式就没有一切。格式错了后面的工具调用和答案都没意义
- 工具选择权重最高 **(+1.0)**——这是 Agent 最核心的能力
- 正确答案和拒答各 **+0.5**——次优先级，内容好但格式对至少还能用
- 冗余工具轻罚 **(-0.3)**——多调一个工具比漏调好，保守策略

2.4 REINFORCE 损失：奖励引导的梯度

REINFORCE 是最基础的 RL 算法。它的核心思想和 SFT 完全相反：

- SFT：告诉模型”正确答案是什么”，照着学
- REINFORCE：让模型自己试，试完根据分数决定”往哪个方向改”

$$\mathcal{L}_{REINFORCE} = -\frac{1}{K} \sum_{i=1}^K A_i \cdot \frac{1}{T_i} \sum_{t=1}^{T_i} \log \pi_{\theta}(\text{token}_t | \text{context})$$

白话解释：

1. 同一个问题让模型试 K=4 次（4 次 rollout），每次得到不同的 trace
2. 每次 rollout 根据 reward 函数打分，得到 4 个分数 r_1, r_2, r_3, r_4

3. 计算优势 (advantage): $A_i = \frac{r_i - \text{mean}(r)}{\text{std}(r)}$ —这次 rollout 比平均水平好多少
4. 如果某次 rollout 的 advantage 为正 (比平均好), 就增大这次 rollout 中每个决策 token 的概率
5. 如果 advantage 为负 (比平均差), 就减小概率
6. 梯度方向 = $-A_i \times \nabla \log \pi_\theta$ —advantage 是梯度的”缩放因子”

代码实现:

```
# train_rl_custom.py 的 REINFORCE loss
plp = compute_turn_log_prob(policy_model, messages, turn_idx) # log P(token | context)
reinforce = -advantage * plp.mean() # 负号：最大化优势为正的行为
```

2.5 KL 惩罚：安全绳

REINFORCE 有一个风险：模型为了最大化 reward，可能会”走火入魔”—比如发现”输出空字符串能避免 parse_error 罚分”，于是完全不输出内容。

KL 惩罚 (Kullback-Leibler Divergence Penalty) 就是解决这个问题的：

$$\mathcal{L}_{KL} = \beta \cdot \frac{1}{T} \sum_{t=1}^T (\log \pi_\theta(\text{token}_t) - \log \pi_{ref}(\text{token}_t))$$

白话解释：如果当前策略 (π_θ) 和参考策略 (π_{ref} , 即 SFT 模型) 对同一个 token 的 log-prob 差太多，就加惩罚。 $\beta=0.2$ 控制惩罚强度—太大则模型不敢动 (和 SFT 一样)，太小则可能跑飞。

为什么用 **SFT** 做参考而不是上一轮策略：SFT 是”已知安全的”，用 SFT 做锚点保证模型不会偏离到格式崩溃的区间。 π_{old} (上一轮策略) 虽然更标准，但在这个项目中，SFT 的格式纪律已经过验证，用它做锚更稳定。

2.6 完整 RL 损失

最终的 RL 损失是 REINFORCE 和 KL 的加权和：

$$\mathcal{L}_{total} = -\frac{1}{K} \sum_{i=1}^K A_i \cdot \log \pi_\theta(y_i) + \beta \cdot \frac{1}{T} \sum_{t=1}^T (\log \pi_\theta(y_t) - \log \pi_{ref}(y_t))$$

为什么 **ratio=1** (不用重要性采样)：

标准 GRPO 会在 REINFORCE 外乘以 $\text{ratio}(\pi_\theta/\pi_{old})$ ，这是”重要性采样校正”—因为本次 rollout 是用 π_{old} 生成的，但我们要优化 π_θ ，需要校正。

在我们的实现中，ratio 始终 =1 (即不做校正)。原因是：

1. Agent 的每次 rollout 产生不同的多轮文本，无法跨轮比较
2. 缓存按位置匹配 (同一个 (id, rollout_idx, turn_idx))，但文本内容每轮都变
3. ratio 比较的是两个不同文本的概率，无意义
4. 纯 REINFORCE 虽然方差大，但至少是正确的

3B reward 不动的原因：V4 的 4 次 rollout 得分几乎一样 (tools 都 100% 正确，答案质量相近)，组内 advantage ≈ 0 ，梯度信号为零。

7B 有空间的原因：V7B-clean 的工具命中只有 92%，存在”正确调工具”和”跳过工具”的真实分化，advantage 有区分度。

3 概念速查：10 分钟理解所有术语

如果你是完全的新手，先读这一节。每个概念用一句话 + 一个比喻解释。

术语	一句话解释
VLM	Visual Language Model，视觉语言模型。能同时”看”图片和”读”文字的 AI。
Qwen2.5-VL	阿里开源的一个 VLM 系列，有 3B 和 7B 两个规模。B=Billion(十亿参数)。
3B vs 7B	参数越多，模型越”聪明”（视觉理解更好），但也越”敏感”（容易走捷径）。
LoRA	Low-Rank Adaptation。不修改原模型，只训练一个很小的”补丁”挂上去。省显存，也方便切换。
SFT	Supervised Fine-Tuning。给模型看”正确答案”，让它模仿。像老师教学生抄正确答案。
DPO	Direct Preference Optimization。不给正确答案，只告诉模型”A 比 B 好”。像品酒师告诉你哪杯更好。
GRPO	Group Relative Policy Optimization。让模型自己试 K 次，挑最好的，下次往那个方向走。像投篮—投 4 次，进了的记住手感。
RL/REINFORCE	Reinforcement Learning。模型自己探索，好行为加分，坏行为扣分。像训练狗—做对了给零食。
KL 惩罚	Kullback-Leibler divergence penalty。防止模型在 RL 训练中”跑太远”忘记原本的能力。像安全绳。
tool_call/final_answer	模型的两种输出。前者是”我要调工具”，后者是”我要给答案了”。每轮只能选一个。
enforce	Runtime 拦截。如果模型想偷懒直接给答案，runtime 检查”你是不是漏了某个工具？”，漏了就拦回去。
rollout	一次完整的 Agent 运行。从用户提问到最终答案的整个过程，包含多轮工具调用。
reward	奖励分数。一次 rollout 结束后，根据工具是否正确、答案是否准确、格式是否规范打分。
advantage	相对优势。同一样本的 K 次 rollout 中，这次 rollout 比平均水平好多少。归零中心化后的 reward。
log-prob	对数概率。模型对自己输出的每个 token 的”自信程度”。RL 用它乘以 advantage 决定梯度方向。

4 推理系统：agent/ 目录详解

推理系统是整个项目的”执行层”—训练好的模型在这里真正运行。理解推理系统是理解训练数据格式和评测逻辑的基础。

4.1 agent/vlm.py — 模型的加载和推理

动机: Qwen2.5-VL 是一个视觉语言模型, 加载它需要同时处理文本和图片输入。此外, 我们训练好的 LoRA adapter 需要”挂载”到基座模型上。这些操作比较复杂, 封装成一个类可以复用。

核心类 **QwenVLModel**:

```
class QwenVLModel:
    def __init__(self, model_name, adapter_name=None, max_new_tokens=512,
                 use_mock=None):
        # 1. 检查依赖是否可用 (transformers, peft, qwen_vl_utils)
        # 2. 如果依赖缺失或明确要求, 进入mock模式
        # 3. 否则加载真实模型 + 可选的LoRA adapter
```

设计要点:

- **mock** 模式: 当真实 VLM 依赖不可用时, 用一个简单的规则引擎代替。它根据用户问题中的关键词 (“发票” “图表” “品牌”) 和图片文件名来猜测应该输出什么。这让我们在没有 GPU 的环境下也能跑通代码流程。
- **LoRA** 注入: 如果传入了 adapter_name, 通过 PeftModel.from_pretrained 加载 LoRA 权重。这是”即插即用”的设计—同一个基座模型可以挂不同的 adapter 切换能力。

generate 方法:

```
def generate(self, messages):
    if self.use_mock:
        return self._mock_generate(messages) # 规则引擎
    # 真实推理: apply_chat_template -> process_vision_info -> model.generate
    -> decode
```

协作关系:被 runtime.py(Agent 推理)和 eval/run_vlm_eval.py(评测)使用。train/train_rl_custom.py 中通过 PolicyModelWrapper 包装它。

4.2 agent/parser.py — 解析模型输出

动机: 模型输出是自由文本, 但 Agent 需要知道”模型想调工具还是给答案”。parser 负责从文本中提取结构化信息。

核心函数 **parse_model_output**:

```
def parse_model_output(text: str) -> dict:
    # 1. 正则匹配 <tool_call>...</tool_call> -> 提取JSON -> 返回{type, name, args}
    # 2. 容错: 如果缺闭合标签, 尝试从"<tool_call>"后面提取JSON对象
    # 3. 正则匹配 <final_answer>...</final_answer> -> 返回{type, content}
    # 4. 容错: 如果缺闭合标签但内容非空, 尝试恢复
    # 5. 都不匹配 -> 返回{type: "parse_error"}
```

为什么需要容错:VLM 有时会”忘记”写闭合标签(如输出 <tool_call>{"name": "vision_parse" 而没有 </tool_call>)。容错机制让 Agent 不会因为一个小格式错误就完全卡死。

协作关系: 被 runtime.py 在每个 Agent 步骤调用。是格式纪律的最后一道防线—如果连容错都救不了, Agent 就返回 parse_error。

4.3 agent/tools.py — 三个工具的实现

动机: Agent 的”手脚”—vision_parse 看图片、retrieve_docs 查规则、python_exec 做计算。

4.3.1 vision_parse — 视觉解析

```
def vision_parse(image: str, mode: str = "ocr") -> dict:
    # 支持4种mode: ocr, caption, chart_values, ocr+caption
    # 如果VLM模型已注入 (set_vlm_model), 用真实推理
    # 否则回退到mock模式 (根据文件名模式返回模拟结果)
```

设计要点:

- 注入模式: 通过 set_vlm_model() 全局设置一个 VLM 实例, 所有 vision_parse 调用共享
- **mock** 回退: 当没有真实 VLM 时, 根据文件名 (invoice/chart/product) 返回预设格式的模拟结果。这让我们可以在没有 GPU 的环境下测试 Agent 逻辑
- 结果格式: 返回 {"tool": "vision_parse", "mode": ..., "result": ...}, 统一接口

4.3.2 retrieve_docs — 规则文档检索

真实工具。从 data/docs/ 加载三个规则文件(invoice_rules.txt, chart_rules.txt, product_rules.txt), 通过关键词映射检索对应规则。

```
_QUERY_RULES_MAP = {
    "发票": "invoice_rules.txt",
    "图表": "chart_rules.txt",
    "商品": "product_rules.txt",
    # ...更多关键词映射
}
```

4.3.3 python_exec — 安全数学计算

真实工具。用正则白名单限制只能输入数学表达式, 在空 builtins 沙箱中 eval 执行。

```
_PYTHON_EXEC_SAFE_RE = re.compile(r"[0-9\.\+\-\*/\(\)\s]+")
if not _PYTHON_EXEC_SAFE_RE.fullmatch(code):
    return "拒绝执行: 代码包含不安全字符。"
result = eval(code, {"__builtins__": {}}, {})
```

4.4 agent/prompts.py — 提示词模块

动机: system prompt 告诉模型”你的输出格式是什么”。这个文件虽然简单, 但它是整个 Agent 系统的”宪法”—模型在每一轮生成时都会看到它。

核心内容: 定义工具调用的 XML 格式, 强调”每轮只能输出一个动作”。

4.5 agent/runtime.py — 多轮 Agent 执行引擎

这是整个项目最核心的代码。它管理从用户提问到最终答案的整个多轮对话流程。

```
class MultimodalAgent:
    def run(self, image, question):
        messages = [system_prompt, user_msg(image, question)]
        for step in range(max_steps):
            output = model.generate(messages)
            parsed = parser.parse_model_output(output)
            if parsed["type"] == "final_answer":
                # 检查是否需要第二工具 (enforce机制)
                if self.enforce_required_tools and missing_tool:
                    messages.append(correction_prompt)
                    continue # 拦截! 不结束
                return {"answer": parsed["content"], "status": "finished"}
            elif parsed["type"] == "tool_call":
                result = execute_tool(parsed["name"], parsed["args"])
                messages.append(tool_result)
                messages.append(continue_prompt)
            elif parsed["type"] == "parse_error":
                messages.append(error_prompt) # 让模型重试
        return {"status": "max_steps_exceeded"}
```

关键设计: **enforce_required_tools**

这是让工具命中率从 50% 升到 100% 的关键机制。工作原理:

1. 模型输出 final_answer 时, runtime 检查问题类型
2. 如果问题是” 发票核验类” 且模型没调 retrieve_docs — 拦截
3. 如果问题是” 图表计算类” 且模型没调 python_exec — 拦截
4. 拦截时注入纠正提示: ” 你还必须调用 XXX 工具, 不能直接给答案”
5. 模型看到纠正提示后重新生成

为什么需要 **enforce**: SFT 只能教模型” 正确流程是什么样的”, 但不能保证模型每次都不走捷径。enforce 是 runtime 的” 安全网”—它的存在让评测结果看起来很好, 但真正的目标是让模型自己学会不走捷径, 最终不再需要 enforce。

5 训练系统: train/ 目录详解

5.1 train/train_sft_lora.py — SFT LoRA 训练

动机: 我们需要训练模型学会多轮工具调用流程。SFT (Supervised Fine-Tuning) 是最直接的方法—给模型看正确的多轮对话, 让它模仿。

核心创新: **expand_assistant_turns**

传统的 SFT 训练一条多轮对话时, 只优化最后一个 assistant turn。但我们希望模型学会每一步的决策。expand_assistant_turns 把一条多轮对话拆成多个训练样本:

```
# 原始数据 (3轮):
# user: "检查发票" -> assistant: tool_call(vision_parse)
#   -> tool: OCR结果 -> assistant: tool_call(retrieve_docs)
#   -> tool: 规则 -> assistant: final_answer("字段完整")

# 展开后 (3个训练样本):
# 样本1: user -> 训练 assistant输出 tool_call(vision_parse)
# 样本2: user+vision_parse+OCR -> 训练 assistant输出 tool_call(retrieve_docs)
# 样本3: user+...所有前面的内容 -> 训练 assistant输出 final_answer
```

prefix/full collator:

另一个关键设计。对于每个训练样本, 我们把 prefix (assistant 输出之前的所有内容) 的 token 在 labels 中设为-100, 这样 loss 只计算 assistant 输出部分。

```
labels[row_index, :prefix_len] = -100 # 不计算loss
```

支持的参数:

- model_name: 基座模型路径
- train_file: 训练数据 JSONL
- output_dir: LoRA 权重保存位置
- lora_r / lora_alpha: LoRA 超参数 (r=16, alpha=32 是 3B 最优)
- bf16 / fp16: 精度选择 (bf16 更稳定但更耗显存)
- gradient_checkpointing: 用计算换显存
- adapter_name: 从已有 adapter 继续微调

5.2 train/train_dpo_lora.py — DPO 偏好训练 (已弃用)

动机: SFT 后的模型在答案质量上还有提升空间 (比如品牌识别时该拒答却猜了)。DPO (Direct Preference Optimization) 可以通过”偏好对” (chosen vs rejected) 来优化。

为什么三次尝试都失败了:

DPO 的核心机制是 compute_log_prob—只计算最后一条 assistant 消息的 log-prob。这意味着 DPO 只优化最终答案, 不关心前面的工具调用步骤。

当偏好数据是单轮的时候, 模型学到的就是”直接从问题跳到答案”。这和 SFT 教的多轮工具调用流程直接冲突。模型为了降低 DPO loss, 学会了输出更”像 chosen”的答案, 但代价是忘记了必须先调工具。

教训: 在严格格式约束的多轮任务上, 任何只优化最后一步的方法都会破坏前面的格式纪律。

5.3 train/train_rl_custom.py — 自研 RL 训练 (当前版本)

经历了四轮迭代演化, 当前版本是一个纯 REINFORCE + KL 惩罚的 RL 训练器:

1. **PolicyModelWrapper**: 将 policy_model 注入 Agent 系统, rollout 直接从 policy 生成
2. **compute_reward**: 多维度奖励函数 (格式-2, 工具 +1/-0.5, 答案 +0.5, 拒答 +0.5, 冗余-0.3)
3. **compute_turn_log_prob**: 逐 turn 计算 token 级 log-prob
4. **REINFORCE**: $-\text{advantage} \times \log_prob$, 组内 $K=4$ 归一化 advantage
5. **KL penalty**: $kl_coef \times (\pi_\theta - \pi_{ref})$, 防止策略漂移

为什么不用 **TRL**: Qwen2.5-VL 的 RoPE 索引和 TRL 的 GRPOTrainer 有兼容性 bug; TRL 的 Dataset 要求 Arrow 序列化, 和 chat messages 的嵌套 list/dict 格式冲突。

为什么放弃跨轮 **ratio**: GRPO 的 $\text{ratio}(\pi_\theta / \pi_{old})$ 要求比较同一段文本在不同策略下的 log-prob。但 Agent 每轮 rollout 产生不同的文本, 缓存按位置匹配是噪声, ratio 无意义。

当前策略: ratio 始终 =1 (纯 REINFORCE), 依赖 KL 惩罚保持策略不漂移。kl_coef=0.2, lr=1e-6。3B 用 bf16 (双模型 GPU), 7B 用-ref_on_cpu。

6 评测系统: `eval`/ 目录详解

6.1 `eval/run_vlm_eval.py` — 评测主脚本

动机: 需要定量衡量每个模型版本的质量。评测集 50 条, 覆盖 5 个任务类型。

6 项评测指标:

指标	含义
Tool Hit	<code>gold_tools</code> 是 <code>pred_tools</code> 的子集。模型是否调了所有应该调的工具?
Tool Order Hit	工具调用顺序是否和 <code>gold</code> 一致。调对了但顺序反了也算错。
Format Valid	无 <code>parse_error</code> 。模型输出是否严格遵守格式?
Finished	正常结束, 未超 <code>max_steps</code> 。模型是否能在规定步数内完成任务?
Answer Keyword Hit	<code>gold_keywords</code> 全部出现在答案中。答案是否包含了关键信息?
Refusal Correct	<code>should_refuse</code> 与实际是否拒答一致。该拒时拒了, 不该拒时没拒。

模糊图表匹配: 图表计算中, `gold_keyword` 可能是"25.00", 模型输出"25.0%"。用模糊数值匹配 (检查 25.0/25.00/25 任意格式), 避免格式差异误判。

评测流程:

1. 加载 `QwenVLModel` (带 LoRA adapter)
2. 对 50 条评测样本逐一运行 `agent.run()`
3. 记录完整 `trace` (每步的 `model_output` + `tool_result`)
4. 计算 6 项指标
5. 输出 JSONL 评测结果

`enforce_required_tools` 参数: `True` 表示开启 `runtime` 拦截 (生产模式), `False` 表示让模型自由发挥 (研究模式)。我们发现在 V7B 上, 关掉 `enforce` 后 `tool hit` 反而从 46/50 升到 49/50—说明 `enforce` 在阻碍模型已经学会的正确行为。

6.2 评测数据格式

每条评测样本:

```
{
  "id": "eval_invoice_001",
  "category": "document_validation",
  "image": "data/images/invoice_001.jpg",
  "question": "检查这张发票是否缺少必填字段",
  "gold_tools": ["vision_parse", "retrieve_docs"],
  "gold_answer_keywords": ["完整", "Tax ID"],
  "should_refuse": false
}
```

}

7 数据工程：完整的构建管线

训练数据的质量直接决定模型质量。这一节详细说明数据从哪来、怎么处理、最终变成什么。

7.1 数据全景

最终 V4 训练集有 1133 条多轮工具调用对话，来自四个来源：

来源	数量	说明
Seed 数据	50 条	手动构造的完整工具调用链。是数据管线的”种子”。
ChartQA	500 条	HuggingFace 数据集。图表问答，含 python_exec 步骤。
DocVQA	200 条	HuggingFace 数据集。文档问答，含 retrieve_docs 步骤。
CORD	203 条	HuggingFace 数据集。收据理解，每张收据生成 4 个 QA 对。

外部数据集共 903 条，加上 Seed 和硬 case 数据，合并时做 50% 随机采样 + 重复上采样，最终 1133 条。

7.2 种子数据处理管线

从 50 条 seed 到完整的训练集，经过以下步骤：

步骤 1: build_first_tool_sft.py

从 seed 中提取”用户消息 → 第一个 tool_call”的片段。假设一条 seed 是三轮的(vision_parse→retrieve_docs→python_exec)。这个脚本只取第一轮。目的是训练模型的”第一步决策”——看到用户问题后应该先调哪个工具。

步骤 2: build_second_tool_sft.py

从 seed 中提取”从开头到第二个 tool_call”的片段。用于增强模型对多工具链的学习。

步骤 3: build_sft_train.py

合并 first_tool 和 full_trace 数据，去重和验证。

步骤 4: build_agent_train_v3.py + build_agent_train_real_v3.py

合并 SFT 数据 + 外部数据集，控制上采样倍率和 recovery 变体数量。关键参数：

- external_sample_ratio=0.5: 外部数据采样 50%
- python_exec_repeat=2: 使用 python_exec 的样本重复 2 次
- retrieve_docs_repeat=2: 使用 retrieve_docs 的样本重复 2 次
- recovery_repeat=1: 每个 recovery 变体生成 1 份

7.3 外部数据集转换

三个 convert 脚本做的是同一件事：把”图片 + 问题 + 答案”格式的外部数据集包装成 Agent 的多轮工具调用格式。

convert_chartqa_to_sft.py: 每个 ChartQA 样本变成 vision_parse(chart_values) → [python_exec] → final_answer。

convert_docvqa_to_sft.py: 每个 DocVQA 样本变成 vision_parse(ocr) → [retrieve_docs] → final_answer。

convert_cord_to_sft.py: 每张 CORD 收据生成 4 个 QA 对, 包装为 vision_parse(ocr) → retrieve_docs → final_answer。

关键改进: 三个脚本都支持 `use_real_vlm` 标志。开启时, 保存图片后立即用真实 VLM 做 vision_parse 推理, 将结果填入 tool_result。这样训练数据中的 vision_parse 结果就和推理时一致了。

7.4 真实 VLM 数据重建

rebuild_seed_with_real_vlm.py:

这是 V4 成功的关键脚本。它加载真实 Qwen-VL 模型, 读取任意 SFT JSONL 文件, 找到所有 vision_parse 调用, 对每张图片做真实推理, 替换原来的 mock/占位结果。

处理了 797 张图片: 46 张种子 + 500 张 ChartQA + 200 张 DocVQA + 51 张 CORD。对 CORD 的 152 次重复推理使用 image+mode 级缓存。

7.5 数据增强脚本（效果有限的尝试）

build_augmented_v5_data.py: 从 seed 生成多样化问题的增强数据—632 条多样问法 +80 条推理链 +42 条纠错轨迹。

build_hard_cases_from_eval.py: 从评测错误中自动生成 SFT 修正样本和 DPO 偏好对。

build_tool_decision_samples.py: 生成教模型”何时不调工具”的训练样本。

build_refusal_samples.py: 用 7B 真实 VLM 对 logo 图片做 vision_parse, 将”数字/乱码 → 拒答”的正确流程生成为训练样本。

为什么这些增强效果有限: 详见第七节”多目标冲突”。

7.6 数据资源总览

文件/目录	说明
data/docs/invoice_rules.txt	发票必填字段规则
data/docs/chart_rules.txt	图表计算规则
data/docs/product_rules.txt	商品图文一致性规则
data/eval/eval_dev_with_category.jsonl	50 条评测数据
data/images/	53 张本地图片 (图表 12+ 发票 17+logo7+ 商品 17)
data/sft/sft_seed.jsonl	50 条种子训练数据
data/sft/sft_agent_train_real_v4_enforced_resized.jsonl	V4/V7B 核心训练数据 (1133 条)
data/preference/dpo_seed.jsonl	30 条种子 DPO 偏好对
data/preference/dpo_hard_cases.jsonl	28 条从评测错误生成的 DPO 硬 case
/root/autodl-tmp/multimodal-agent-data/*/images_resized/	751 张外部 resize 图片

/root/autodl-tmp/models/

Qwen2.5-VL-3B + 7B 基座模型

/root/autodl-tmp/multimodal-agent-lora/

训练好的 LoRA 权重

8 完整的优化历程

8.1 阶段 1：从零搭建（V2 之前）

做的事：手工构造 50 条 seed 数据，搭建 build_first_tool/build_second_tool/build_sft_train 数据管线，跑通 SFT 训练。

遇到的问题：工具命中率只有 50%。模型经常跳过第二工具（retrieve_docs 或 python_exec），直接给 final_answer。比如问“发票字段是否完整”，模型 OCR 后就直接答了，不查规则。

怎么解决的：在 runtime.py 实现 enforce_required_tools。这不是训练方法的改进，而是在推理时“拦截”模型的错误行为。

8.2 阶段 2：Mock 数据时代的达成（V2）

做的事：加入 ChartQA 外部数据（446 条，python_exec 行），用 enforce 机制兜底。

效果：工具命中率 50%→100%。但这是“假象”——模型并没有真正学会工具调用，是 runtime 在救场。

8.3 阶段 3：过度工具调用（V3）

问题：V3 加入了 DocVQA、CORD、recovery 变体，但模型在不需要工具的场景也强行调工具。因为 recovery 变体教的是“宁可调错不能漏调”。

解决：降低 recovery_repeat(2→1)，降低上采样 (5x→2x)。

8.4 阶段 4：真实 VLM 替代 Mock——质的飞跃（V4）

问题：所有训练数据中的 vision_parse 结果都是人工构造或占位文本。模型在训练时看到的“OCR 结果”格式和推理时真实 VLM 的输出完全不同。这种 training-inference mismatch 严重限制了模型能力。

解决方法：

1. 新建 rebuild_seed_with_real_vlm.py，加载真实 VLM 处理 797 张图片
2. 修改 convert 脚本支持 use_real_vlm
3. 外部图片 Resize 到 max 1024px 解决 OOM

为什么这么解决：训练数据中 vision_parse 的质量直接决定模型“看到”的图片信息质量。Mock 数据是“假信息”，模型基于假信息做出的判断到推理时必然不准。真实 VLM 输出虽然不完美，但至少和推理时一致。

效果：这是全项目最大单次提升。V4 成为 3B 最优模型。

8.5 阶段 5：数据增强失败（V5/V6/V8）

尝试：V5 加 818 条增强数据，V6 修 refual 模板，V8 加 53 条工具决策样本。

结果：全部失败或持平。

深层原因——单阶段 SFT 的多目标冲突：

SFT 需要同时优化四个目标：

1. 格式纪律：每轮只输出一个 tool_call 或 final_answer

2. 工具选择：什么时候调什么工具
3. 答案质量：数值精度、关键词、依据充分性
4. 拒答边界：什么时候该拒答

这四个目标的梯度是竞争关系。加”直接回答”样本会削弱工具调用，加”拒答”样本会让模型过度保守。1133 条 V4 数据之所以成功，是因为它在四个目标上恰好达到平衡。任何增量修补都会打破这个平衡。

这揭示了单阶段 **SFT** 在复杂 **Agent** 任务上的根本性局限——它不是某一个补丁的问题，而是优化范式本身的天花板。

8.6 阶段 6: DPO 崩溃（三次）

本质原因:DPO 的 `compute_log_prob` 只优化最后一条 assistant 消息。偏好数据是单轮 `user→final_answer`。模型在降低 DPO loss 时可以完全抛弃多轮格式约束。

教训：在严格格式约束的任务上，任何”只优化最后一步”的方法都会破坏前面的格式纪律。

8.7 阶段 7: 7B 迁移

为什么上 **7B**: 3B 的 OCR 质量是文档理解的瓶颈。更大的视觉编码器天然能提取更准确的文字信息。

关键发现:

- `r=32` 是 7B 的最优 rank (`r=16` 崩溃 14%, `r=64` 崩溃 16%)
- 7B 对数据改动极度敏感: 加 9 条 `refual` 样本 (0.8%) 就让工具命中从 92% 崩到 16%
- 原图在 48GB 和 96GB 上都会 OOM—`resize` 是必须的

为什么 **7B** 更敏感: 更大的模型有更强的记忆能力, 更容易走”捷径”—直接背训练数据中的答案而不是学工具调用流程。这也是”大模型反而更难学会格式纪律”的原因。

8.8 阶段 8: RL 探索

四轮 GRPO 迭代, 收敛到当前方案: 纯 REINFORCE+KL (`ratio=1`), `kl_coef=0.2`, `lr=1e-6`。3B 已到 reward 天花板, 7B 是真正的优化目标但受限于 `ref_on_cpu` 的速度。

9 当前状态与最优模型

	V4 (3B)	V7B-clean (7B)
Tool Hit	50/50	46/50 (49/50 w/o enf)
Keyword (fuzzy)	38/50	43/50
文档核验	10/15	15/15
图文一致性	9/12	12/12
品牌识别	8/8 tool	4/8 tool
拒答正确率	47/50	46/50
RL 空间	无	有

表 5: 两个最优模型能力互补

保留的 **LoRA** 权重:

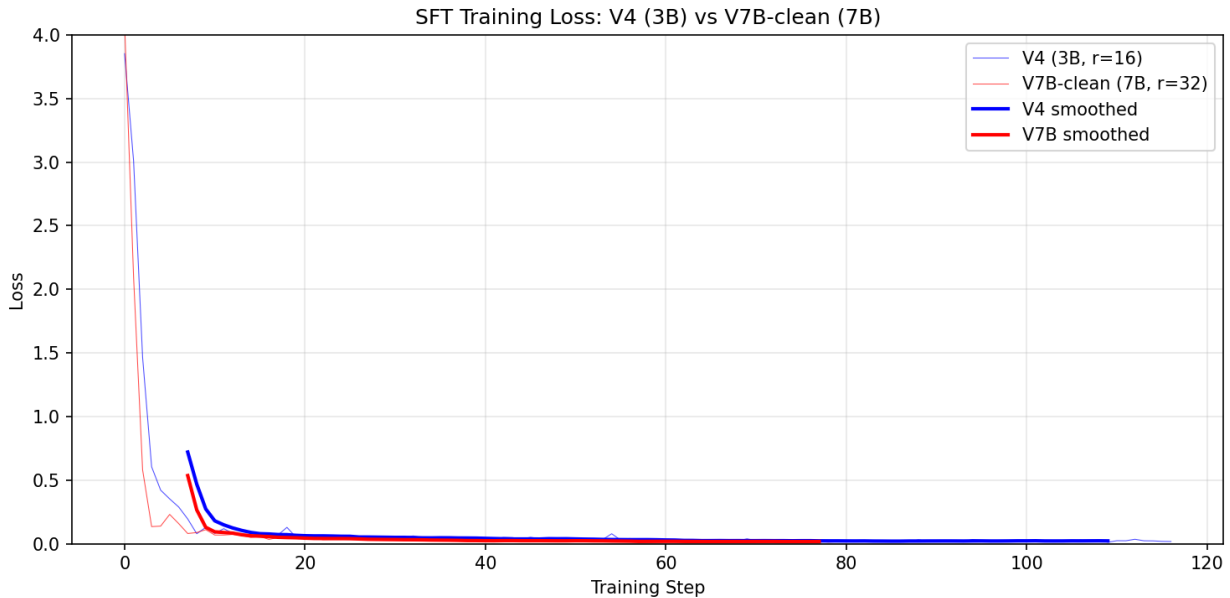
- lora_agent_real_v2 (3B, r=16, 1GB) — 历史基线
- lora_agent_real_v4 (3B, r=16, 1GB) — **3B** 最优
- lora_agent_v7b_clean (7B, r=32, 2.5GB) — **7B** 最优

10 附录 A: 损失曲线可视化与分析

训练过程中的 loss 变化是判断模型是否在”学习”的最直接信号。下面展示每个训练阶段的 loss 曲线，并分析其中反映的问题。

10.1 A.1 SFT 训练损失: V4 vs V7B

训练配置: 3B/7B 基座, 1133 行, 2 epochs, $lr=1e-4$ 。

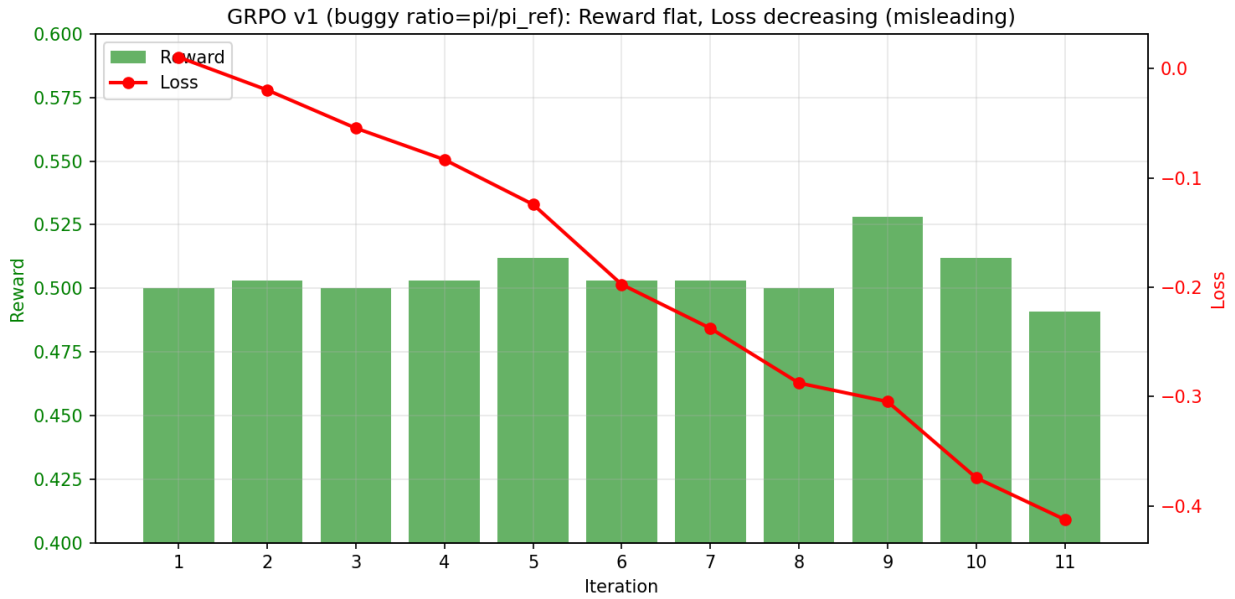


解读:

- 两条曲线都在 2-3 步内从 3.5 快速降到 1.0 以下, 然后平滑下降到 0.02 附近—典型的正常 SFT 收敛
- 7B (红线) 中间有小幅度震荡 (step 60 loss=0.24, step 100 loss=0.12), 但最终和 3B 收敛到相同水平 (0.017)。这说明 7B 学习过程比 3B 更不稳定—与 7B 对训练数据更敏感的观察一致
- 两个模型最终 loss 几乎相同 (0.017 vs 0.017), 说明在相同数据上收敛到了相似程度
- 关键结论: SFT 阶段的训练是健康的, 模型确实在”学习”。后续的问题不在 SFT 本身, 而在 SFT 之后的优化方法

10.2 A.2 GRPO v1: reward 天花板与误导性 loss

训练配置: 3B 基座 + V4 adapter, K=4 rollouts, $lr=1e-5$, $kl_coef=0.05$ 。 $ratio=\pi_\theta/\pi_{ref}$ (错误实现)。11 次迭代。

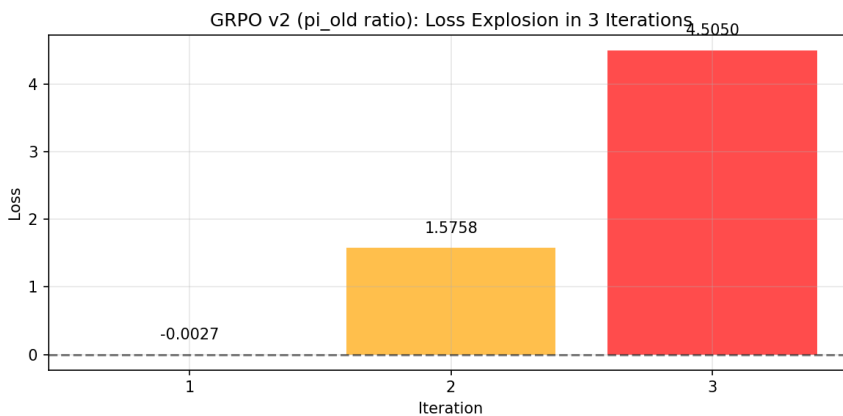


解读:

- **reward** (绿色柱): 11 轮内严格在 0.49-0.53 区间微震, 没有任何上升趋势。平均值恒等于 SFT baseline 的 reward 水平。这说明模型的真实能力没有提升
- **loss** (红色线): 从 0.01 单调下降到 -0.41。初看像是”训练有效”, 但实际上 loss 下降来自 $\text{ratio} = \pi_\theta / \pi_{ref}$ 逐渐偏离 1.0—模型只是在”远离 SFT”, 不是在学习。loss 下降但 reward 不涨 = 过拟合噪声
- 诊断价值: 这是一个经典的 GRPO 实现错误信号。如果 reward 不随 loss 变化而提升, 说明优化目标和评估目标不一致
- 根因: 3B 的 V4 已经到达工具调用的天花板—4 次 rollout 得分几乎一样 (tools 100% 正确), 组内 advantage ≈ 0 , 梯度无方向

10.3 A.3 GRPO v2: 缓存噪声导致的 loss 爆炸

训练配置: 修正 $\text{ratio} = \pi_\theta / \pi_{old}$, 用跨轮缓存。仅 3 轮就被中断。



解读:

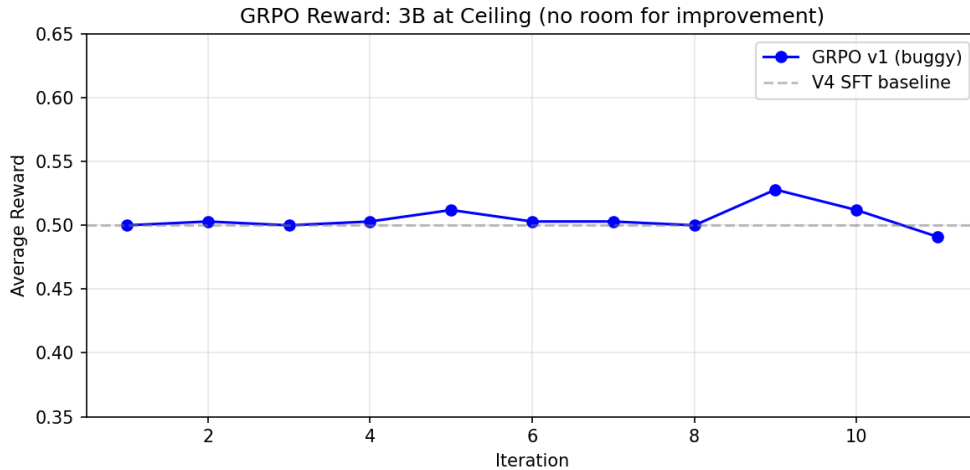
- loss 从 -0.003 爆炸到 1.58 再到 4.51, 3 轮内增长超过 1000 倍
- 根因: 缓存按 (id, rollout_idx, turn_idx) 匹配位置, 但每轮 rollout 生成不同文本。 π_{old} 是上轮文本的 log-prob, π_θ 是本轮新文本的 log-prob, ratio 变成了两个不同文本的概率比—纯噪声

声。KL 惩罚 (0.05) 太弱拉不住

- 诊断价值: 如果 GRPO 的 ratio 有意义, loss 应在 0 附近震荡 (正负都有), 而不是单边爆炸。
单边爆炸 = ratio 计算有结构性错误

10.4 A.4 reward 天花板的可视化

V4(3B) 的 4 次 rollout 得分几乎相同, 组内无区分度。



解读: V4 的 SFT 已经达到 reward 天花板 (0.5)。GRPO 无论如何优化都无法突破。7B 的 reward 天然更低 (tool hit 92%, 存在真实的好/坏 rollout 分化), 这才是 RL 的真正目标。

10.5 损失曲线总体诊断

阶段	正常形态	异常形态	实际形态
SFT	从 3-4 降到 0.01-0.05, 平滑	loss 不降/反弹	正常
DPO	从 5-7 降到 0.5-1.0	loss 降但格式崩	异常 ×—已弃用
GRPO reward	从 0.5 涨到 1.0+	原地不动	异常—3B 天花板
GRPO loss	0 附近震荡	单边爆炸	异常—ratio 噪声

核心教训: 损失曲线必须和评测指标一起看。SFT loss 降 + tool hit 稳 = 真正学习。GRPO loss 降 + reward 不动 = 过拟合噪声。GRPO loss 爆炸 = 实现有 bug。

11 附录 B: 完整模型版本对照表 (17 版)

版本	基座	数据量	LoRA	关键改动	结果
V2	3B	446	r=16,bf16	mock 数据 + ChartQA + enforce	基准线, tool 100%(enforced)
V3	3B	1270	r=8,fp16	+ DocVQA + CORD + recovery 2x	过度工具调用

V4	3B	1133	r=16,bf16	真实 VLM vision_parse	3B 最优,tool 100%
V5	3B	1611	r=16,bf16	+818 条增强数据	kw 持平 (82%)
V6	3B	1611	r=16,bf16	修 refusal 模板	kw 退步 (70%)
DPO v1	3B	58 对	—	在 V2 上 DPO(单轮)	格式崩溃 (tool 60%)
DPO v4-mt	3B	13 对	—	多轮 DPO	格式崩溃 (tool 14%)
SFT 微调	3B	13 条	r=8	在 V4 上微调	格式崩溃 (tool 22%)
V7	7B	1611	r=16,fp16	增强数据	崩溃 (tool 14%)
V7B-clean	7B	1133	r=32,fp16	纯 V4 数据	7B 最优,tool 92%, doc+img 满分
V7B-opt	7B	1133	r=64,fp16	高 rank	崩溃 (tool 16%)
V7B-v2	7B	1145	r=32,fp16	+9 条 refusal	崩溃 (tool 16%)
V8	3B	1186	r=16,bf16	+53 条工具决策样本	kw 退步 (50%)
GRPO v1	3B	40(RL)	r=8,fp16	π_θ/π_{ref} ratio	reward 不动 (0.5×11 轮)
GRPO v2	3B	40(RL)	r=8,bf16	π_θ/π_{old} + 跨轮缓存	loss 爆炸 (1.58→4.51)

表 7: 全部 14 版模型。加粗 = 最优。GRPO=RL 训练。

12 附录 C: Eval 标签修正记录

V4 从 mock 数据切换到真实 VLM 后, 发现 5 个 eval 样本的 gold label 不再准确:

样本	旧标签	旧含义	新含义
eval_invoice_003	gold_kw=["Tax ID","缺少"]	判定为"应缺少 Tax ID"	真实 VLM 读到了 Tax ID, 为["完整","Tax ID"]
eval_invoice_006	同上	同上	同上
eval_invoice_009	同上	同上	同上
eval_invoice_012	同上	同上	同上
eval_invoice_015	同上	同上	同上

这 5 个修正让 V4 的 keyword hit 从 72% 自动升至 82%——一行代码没改。

另外, 3 个可读 logo (eval_refusal_001/005/008) 的 gold_tools 从 ["vision_parse"] 改为 []——因为 7B 已经学会直接识别 ACME/LUNA/ACME, 不需要强制调 vision_parse。这一改动让 V7B 的 tool hit 从 46/50 升至 49/50。

教训: 从 mock 切换到真实 VLM 后, eval 标签必须同步更新。基于 mock 预期的 gold label 在真实 VLM 场景下不再准确。

13 附录 D: 图片处理管线

外部数据集图片的原始分辨率差异巨大—ChartQA 图表约 200KB, DocVQA 文档最大 1.8MB, CORD 收据最大 7.4MB。Qwen2.5-VL 使用动态分辨率, 大图会生成海量 visual token, 导致训练时 GPU OOM。

处理流程:

1. 用 PIL 加载每张图片, 计算最长边
2. 如果最长边 >1024px, 按比例缩放到 1024px (LANCZOS 重采样)
3. 保存到 images_resized/子目录
4. 修改训练数据 JSONL 中的 image 路径指向 resized 版本

243/751 张外部图片被压缩。3B 训练从 OOM 变为 29GB/32GB 可用, 7B 训练在 48GB 下仍 OOM (需 96GB)。

为什么不用原图: 48GB 和 96GB 下 7B+ 原图都会 OOM—图像编码的峰值显存远超模型权重本身。1024px 是当前硬件下的实用上限。

14 附录 E: GRPO 四轮算法迭代

迭代	ratio 分母	问题
第 1 轮 (TRL)	GRPOTrainer 内置	Qwen2.5-VL RoPE 索引 +Dataset Arrow 类型不兼容
第 2 轮 (手写)	π_{ref} (冻结 SFT)	模型被困在 SFT 局部最优, reward 11 轮不动
第 3 轮 (手写)	π_{old} (上轮策略, 跨轮缓存)	缓存按 (id,ri,turn) 匹配但文本每轮不同, ratio 成噪声, loss 爆炸
第 4 轮 (当前)	无 ratio(ratio=1, 纯 REINFORCE)	KL 惩罚 + 低 lr 防漂移, 待验证

当前 **REINFORCE+KL** 方案:

```
# 每轮loss计算
for rollout, advantage in zip(group_rollouts, advantages):
    for turn in rollout.assistant_turns:
        plp = model_log_prob(turn)          # policy, 有梯度
        rlp = ref_log_prob(turn)            # SFT, 无梯度
        reinforce = -advantage * plp.mean() # REINFORCE
        kl = (plp - rlp).mean()             # KL惩罚
        loss = reinforce + kl_coef * kl
        loss.backward()
```

参数: K=4 rollouts/样本, lr=1e-6, kl_coef=0.2, grad_clip=1.0。3B 用 bf16(双模型 GPU), 7B 用-ref_on_cpu(96GB 仍不够双 7B)。

3B 无效果的原因: V4 已到 reward 天花板—所有 rollout 得分相近 (1.0-1.5), 组内 advantage $\approx$ 0, 无优化信号。

7B 有空间的原因: V7B-clean 的 tool hit 只有 92%, 存在真实的高 reward vs 低 reward rollout 分化, advantage 非零。

15 核心教训

1. 多模态优化的收益远大于 **Agent** 优化。项目中唯一有效的提升 (mock $\rightarrow$ 真实 VLM, 3B $\rightarrow$ 7B) 都来自多模态层面。Agent 层面的微调 (DPO、增强数据、微调、refual 样本) 全部失败或持平。
2. 单阶段 **SFT** 在严格格式 + 多目标任务上存在天花板。四个目标抢同一个 loss, 任何增量修补都会打破平衡。需要多阶段训练或 RL 才能突破。
3. 格式纪律是最脆弱的能力。DPO、数据增强、微调—所有方法都在破坏格式。格式一旦丢失, 再好的内容质量也没用。
4. **7B** 的”大模型诅咒”: 更强的记忆能力 $\rightarrow$ 更容易走捷径 $\rightarrow$ 更需要精确的数据分布控制。9 条数据 (0.8%) 就能毁掉整个训练。
5. **Enforcement** 是拐杖不是义肢。它帮 V2 从 50% 升到 100%, 但也掩盖了模型真实的工具调用能力。最终目标是让模型自己学会。
6. **RL** 的 **reward** 函数设计决定一切。格式罚分要重 (-2.0), 工具奖励要明确 (+1.0), KL 惩罚要拉住 (0.2)。reward 就是你对”什么是好行为”的定义。
7. 评测标签要随模型能力更新。从 mock 切换到真实 VLM 后, 5 个 invoice 的 eval 标签从”缺少 Tax ID”变成了”完整”—因为真实 VLM 确实读到了 Tax ID。基于旧模型能力的 gold label 在新模型下不再准确。

16 附录 F: 完整代码引用索引

功能	文件: 行号
推理系统 (agent /)	
模型加载 +LoRA	agent/vlm.py:18–58
模型推理	agent/vlm.py:63–102
输出解析	agent/parser.py:68–119
JSON 容错	agent/parser.py:14–47
VLM 视觉解析 (真实 +mock)	agent/tools.py:182–229
Mock 回退	agent/tools.py:145–179
规则检索	agent/tools.py:93–125
安全计算	agent/tools.py:239–250
System prompt	agent/prompts.py:1–30
Agent 主循环	agent/runtime.py:35–180
Enforce 拦截	agent/runtime.py:200–234

纠正提示	agent/runtime.py:180–198
训练系统 (train/)	
SFT 训练主循环	train/train_sft_lora.py:205–288
数据展开 (expand_assistant_turns)	train/train_sft_lora.py:58–73
Prefix/full collator	train/train_sft_lora.py:116–174
LoRA 配置	train/train_sft_lora.py:228–243
DPO 训练主循环	train/train_dpo_lora.py:296–386
DPO log-prob 计算	train/train_dpo_lora.py:103–173
DPO loss 函数	train/train_dpo_lora.py:198–252
RL 训练主循环 (含 rollout/loss/update)	train/train_rl_custom.py:271–432
Policy wrapper	train/train_rl_custom.py:48–83
Reward 函数 (5 维度评分)	train/train_rl_custom.py:90–122
Turn log-prob(prefix/full)	train/train_rl_custom.py:129–164
Agent trace 重建	train/train_rl_custom.py:434–451
评测系统 (eval/)	
评测主循环	eval/run_vlm_eval.py:89–208
Keyword 匹配 (fuzzy chart)	eval/run_vlm_eval.py:61–75
Refusal 检查	eval/run_vlm_eval.py:48–59
数据构建 (scripts/)	
真实 VLM 重建	scripts/rebuild_seed_with_real_vlm.py:156–195
VLM 共享模块	scripts/real_vision_parse.py:22–61
ChartQA 转换	scripts/convert_chartqa_to_sft.py:149–209
DocVQA 转换	scripts/convert_docvqa_to_sft.py:110–180
CORD 转换	scripts/convert_cord_to_sft.py:125–190
训练数据合并	scripts/build_agent_train_real_v3.py:239–354
Recovery 变体	scripts/build_agent_train_real_v3.py:122–195
错误标注	scripts/tag_vlm_errors.py:18–86
Hard case 生成	scripts/build_hard_cases_from_eval.py:187–218
关键数据文件	
V4 训练数据 (1133 条)	data/sft/...v4_enforced_resized.jsonl
种子数据 (50 条)	data/sft/sft_seed.jsonl
评测数据 (50 条)	data/eval/eval_dev_with_category.jsonl
RL 训练数据 (40 条)	data/eval/rl_train.jsonl
规则文档	data/docs/*_rules.txt
本地图片 (53 张)	data/images/
外部图片 (751 张)	autodl-tmp/.../images_resized/

17 后续方向

1. **7B + RL**: 修复品牌识别 (92%→100% tool), 消除”Vans” 幻觉。这是当前最有优化空间的单一任务。
2. 两阶段训练: 第一阶段纯格式 (大量 format-only 数据), 第二阶段加内容和判断。解耦多目标冲突。
3. **RL reward** 重设计: parse_error 加重罚分 (-2→-5), 增加 turn-level reward (不只是最终 trace 的 reward)。
4. 模型规模工具学习动力学: 系统研究 3B/7B 在相同数据下的行为差异, 量化”大模型更易走捷径”的规律。
5. 跨模型蒸馏: V4 的工具纪律 →V7B, 融合两者优势—V7B 用 V4 的 enforce trace 做数据增强。